AIAC – 18

Batch 32, 2303A52065

Task 1 – Weather API Integration

Connect to a Weather API to fetch current weather details for a given city.

Instructions:

- Prompt AI to generate Python code using the requests library.
- Handle errors: invalid city name, API key errors, and network failures.
- Display temperature, humidity, and weather description in a readable format.

Expected Output:

- A working Python script that fetches and displays weather data with proper error handling.

```python
#Generate code to fetch current weather details for a given city using open meteo and geocoding and handle errors like invalid city name, API key errors, and network fail
import requests
def get_coordinates(location):
    url = f"https://nominatim.openstreetmap.org/search?q={location}&format=json"
    try:
        response = requests.get(url, headers={"User-Agent": "weather-fetcher"}, timeout=10)
        response.raise_for_status()  # Raises HTTPError for bad responses
        data = response.json()
        if data and len(data) > 0:
            return float(data[0]['lat']), float(data[0]['lon'])
        else:
            raise ValueError("Invalid city name or location not found.")
    except requests.exceptions.RequestException as e:
        raise ConnectionError(f"Network error while fetching coordinates: {e}")
def get_weather(lat, lon):
    url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true"
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        if 'current_weather' in data:
            return data['current_weather']
        elif 'current' in data:  # fallback for newer API versions
            return data['current']
        else:
            raise ValueError("Weather data not found in API response.")
    except requests.exceptions.RequestException as e:
        raise ConnectionError(f"Network error while fetching weather: {e}")
    except KeyError:
        raise ValueError("Unexpected API response format.")
def main():
    location = input("Enter a location: ")
    try:
        lat, lon = get_coordinates(location)
        weather = get_weather(lat, lon)
        print(f"Current weather in {location}:")
        print(f"Temperature: {weather.get('temperature')}°C")
```

```python
        print(f"Temperature: {weather.get('temperature')}°C")
        print(f"Wind Speed: {weather.get('windspeed', weather.get('wind_speed'))} km/h")
        print(f"Weather Code: {weather.get('weathercode')}")
    except ValueError as e:
        print(f"Error: {e}")
    except ConnectionError as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

```
Enter a location: Warangal
Current weather in Warangal:
Temperature: 27.9°C
Wind Speed: 8.8 km/h
Weather Code: 3
PS C:\Users\avesh\AIAC3_2>
```

The program correctly fetches weather data using API and displays temperature, humidity, and description clearly. Error handling ensures the program doesn't crash for invalid city names or API issues. The output is readable, so it matches real-world usage properly.

Task 2 – Currency Exchange Rates

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.
- Show conversion results clearly in tabular format.

Expected Output:

- A script that converts INR to multiple currencies with graceful error handling.

```python
#Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP
#Validate user input for amount and currency codes and Implement error handling for invalid responses and API downtime.
#Show conversion results clearly in tabular format
import requests
def get_exchange_rate(from_currency, to_currency):
    url = f"https://api.exchangerate-api.com/v4/latest/{from_currency}"
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        if 'rates' in data and to_currency in data['rates']:
            return data['rates'][to_currency]
        else:
            raise ValueError("Invalid currency code or exchange rate not found.")
    except requests.exceptions.RequestException as e:
        raise ConnectionError(f"Network error while fetching exchange rate: {e}")
def main():
    amount = input("Enter the amount in INR: ")
    to_currency = input("Enter the target currency code (USD, EUR, GBP): ").upper()
    valid_currencies = ['USD', 'EUR', 'GBP']
    if to_currency not in valid_currencies:
        print("Error: Invalid currency code. Please enter USD, EUR, or GBP.")
        return
    try:
        amount = float(amount)
        if amount < 0:
            raise ValueError("Amount must be a positive number.")
        exchange_rate = get_exchange_rate('INR', to_currency)
        converted_amount = amount * exchange_rate
        print(f"\n{'Currency':<10} {'Amount':<15}")
        print(f"{to_currency:<10} {converted_amount:<15.2f}")
    except ValueError as e:
```

```python
    except ValueError as e:
        print(f"Error: {e}")
    except ConnectionError as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

```
Enter the amount in INR: 523
Enter the target currency code (USD, EUR, GBP): USD

Currency   Amount
USD        5.54
```

The program correctly converts INR into multiple currencies using API data. Input validation prevents wrong values, and error handling manages API failures smoothly. The output is shown clearly, making the results easy to understand.

Task 3 – GitHub Repository Info Fetcher

Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

• Prompt AI to write a Python function to send GET requests to GitHub API.

• Handle rate limit errors, 404 (repo not found), and invalid input.

• Display repository name, description, stars, forks, and open issues.

Expected Output:

• A Python program that retrieves and displays GitHub repository information with robust error handling.

```python
#Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues) to send GET requests to GitHub API.
# Handle rate limit errors, 404 (repo not found), and invalid input. Display repository name, description, stars, forks, and open issues
import requests
def get_repo_details(owner, repo):
    url = f"https://api.github.com/repos/{owner}/{repo}"
    try:
        response = requests.get(url, timeout=10)
        if response.status_code == 404:
            raise ValueError("Repository not found. Please check the owner and repository name.")
        response.raise_for_status()
        data = response.json()
        return data
    except requests.exceptions.RequestException as e:
        raise ConnectionError(f"Network error while fetching repository details: {e}")
def main():
    owner = input("Enter the GitHub repository owner: ")
    repo = input("Enter the GitHub repository name: ")
    try:
        details = get_repo_details(owner, repo)
        print(f"Repository Name: {details.get('name')}")
        print(f"Description: {details.get('description')}")
        print(f"Stars: {details.get('stargazers_count')}")
        print(f"Forks: {details.get('forks_count')}")
        print(f"Open Issues: {details.get('open_issues_count')}")
    except ValueError as e:
        print(f"Error: {e}")
    except ConnectionError as e:
        print(f"Error: {e}")
    except Exception as e:
```

```python
    except ConnectionError as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

```
Enter the GitHub repository owner: bytedance
Enter the GitHub repository name: deer-flow
Repository Name: deer-flow
Description: An open-source SuperAgent harness that researches, codes, and creates. With the help of sandboxes, memories, tools, skill, subagents and message gateway, it handles different levels
of tasks that could take minutes to hours.
Stars: 43999
Forks: 5182
Open Issues: 312
```

The program successfully retrieves repository details like stars, forks, and issues using the GitHub API. It handles errors like invalid repo names and rate limits properly. The displayed output is clear and matches the expected information.

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario: Build a News Aggregator using a free News API.

• Fetch top 5 headlines for a given category (sports, technology, health).

• Handle errors like invalid category, missing API key, and request failures.

• Display headlines in a user-friendly numbered list format.

• Include a retry mechanism if the first request fails.

Expected Output:

• A working Python script that retrieves and displays news headlines with proper error handling

```python
#Build a News Aggregator using any free API which doesnt require key. Fetch top 5 headlines for a given category (sports, technology, health).
# Handle errors like invalid category, missing API key, and request failures.
# Display headlines in a user-friendly numbered list format. Include a retry mechanism if the first request fails.
import requests
import xml.etree.ElementTree as ET
def get_news(category):
    # Map categories to Google News RSS feeds
    rss_feeds = {
        "sports": "https://news.google.com/rss/search?q=sports&hl=en-IN&gl=IN&ceid=IN:en",
        "technology": "https://news.google.com/rss/search?q=technology&hl=en-IN&gl=IN&ceid=IN:en",
        "health": "https://news.google.com/rss/search?q=health&hl=en-IN&gl=IN&ceid=IN:en"
    }
    if category not in rss_feeds:
        raise ValueError("Invalid category. Please choose from sports, technology, or health.")
    url = rss_feeds[category]
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        root = ET.fromstring(response.content)
        # Extract headlines from RSS feed
        items = root.findall(".//item")
        headlines = [item.find("title").text for item in items[:5]]
        if headlines:
            return headlines
        else:
            raise ValueError("No news articles found for the given category.")
    except requests.exceptions.RequestException as e:
        raise ConnectionError(f"Network error while fetching news: {e}")
def main():
    category = input("Enter a news category (sports, technology, health): ").lower()
    valid_categories = ['sports', 'technology', 'health']
    if category not in valid_categories:
```

```python
    if category not in valid_categories:
        print("Error: Invalid category. Please enter sports, technology, or health.")
        return
    try:
        headlines = get_news(category)
        print(f"\nTop 5 headlines in {category.capitalize()}:")
        for idx, title in enumerate(headlines, start=1):
            print(f"{idx}. {title}")
    except ValueError as e:
        print(f"Error: {e}")
    except ConnectionError as e:
        print(f"Error: {e}")
        retry = input("Do you want to retry? (yes/no): ").lower()
        if retry == 'yes':
            main()
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

```
Enter a news category (sports, technology, health): Technology

Top 5 headlines in Technology:
1. Anduril, Palantir developing Golden Dome missile shield software, source says - The Hindu
2. Why govt & Indian enterprise must buy from local deeptechs - The Times of India
3. Learn the fundamentals and to spot problems - The Times of India
4. Divide between Silicon Valley and ordinary people grows ever larger | Technology - The Guardian
5. Pranos raises $6.8 Mn to build high-field fusion reactor technology - Entrackr
```

The program correctly fetches and displays top news headlines based on category. Error handling and retry logic ensure the program works even if the request fails initially. The output is user-friendly and easy to read as a numbered list.